Local-Splitter: A Measurement Study of Seven Tactics for Reducing Cloud LLM Token Usage on Coding-Agent Workloads

Jay Lux Ferro
SPERIXLABS
jay@sperixlabs.org

April 12, 2026

Abstract

We present a systematic measurement study of seven tactics for reducing cloud LLM token usage when a small local model can act as a triage layer in front of a frontier cloud model. The tactics are: (1) local routing, (2) prompt compression, (3) semantic caching, (4) local drafting with cloud review, (5) minimal-diff edits, (6) structured intent extraction, and (7) batching with vendor prompt caching. We implement all seven in an open-source shim that speaks both MCP and the OpenAI-compatible HTTP surface, supporting any local model via Ollama and any cloud model via an OpenAI-compatible endpoint. We evaluate each tactic individually, in pairs, and in a greedy-additive subset across four coding-agent workload classes (edit-heavy, explanation-heavy, general chat, RAG-heavy). We measure tokens saved, dollar cost, latency, and routing accuracy. Our headline finding is that *T1 (local routing) combined with T2 (prompt compression) achieves 45–79% cloud token savings on short-output workloads, while on RAG-heavy workloads the full set including T4 (draft-review) achieves 51% savings.* We observe that the optimal tactic subset is workload-dependent, which we believe is the most actionable finding for practitioners deploying coding agents today.

1 Introduction

Cloud LLM API costs have become a significant line-item in the operational budget of developer-tooling companies [7]. Coding agents [11, 14] now routinely send tens of thousands of tokens to a frontier model per task, most of which are repetition, boilerplate, or context that could be cheaper to produce locally. Meanwhile, local-inference-ready small models of the 1–7 B parameter class have advanced to the point where they can correctly answer a meaningful fraction of developer queries on consumer hardware [1, 9, 15].

This paper asks: *given a local small model and a frontier cloud model, what is the best way to split work between them to minimise total cloud token usage while keeping response quality near the baseline?* We survey seven concrete tactics that each attack a distinct source of token waste, implement all seven, and evaluate them individually and in combination on four realistic workload classes.

Our contributions are:

- **A taxonomy of seven orthogonal tactics** for reducing cloud token usage via a local triage model (§3).
- **A reference implementation** that is vendor-agnostic at both ends — any Ollama-compatible local model and any OpenAI-compatible cloud endpoint — and exposes both an MCP interface and an OpenAI-compatible HTTP proxy (§4).
- **An empirical evaluation** measuring per-tactic and per-combination savings, latency, and quality deltas across four workload classes and multiple model pairs (§5, §6).

- **The observation that optimal tactic subset is workload-dependent**, with concrete recommendations by workload class (§7).
- **A release of code, workloads, and evaluation harness** for reproduction and extension.

2 Related Work

Speculative decoding. Leviathan *et al.* [12] introduced speculative decoding as a token-level acceleration technique in which a small draft model proposes several tokens and a large verifier accepts or rejects them. Medusa [6] extends this with multiple decoding heads. Our tactic T4 adopts the same structural idea at the *application* layer: the local model drafts a complete response, and the cloud model is asked to review or patch it rather than generate from scratch.

Prompt compression. LLMLingua [10] and its successor LLMLingua-2 [19] compress long input contexts by identifying and removing low-importance tokens. Our tactic T2 is a semantic-compression variant that runs the local model as a text-to-text compressor rather than a token-level filter.

Semantic caching. GPTCache [5] caches responses keyed by embedding similarity. We replicate this as tactic T3 and extend it with rigorous quality measurement that the original did not report.

Routing and cascades. FrugalGPT [7] and RouteLLM [17] present model-cascade architectures where requests are routed to progressively larger models based on confidence. Our tactic T1 is a single-stage local/cloud router, and we compare our empirical savings to those reported in the cascade literature.

Vendor prompt caching. Anthropic's `cache_control: ephemeral` [2] and OpenAI's automatic prompt caching [18] enable substantial discounts when stable prefixes are re-sent. These are strictly cloud-side features; our tactic T7 integrates them into the splitter's outbound requests.

What we add. Individual tactics have been studied in isolation, often with different benchmarks. To our knowledge, this paper is the first to measure all seven on a common benchmark, to measure their pairwise interactions, and to quantify the workload-dependence of the optimal subset.

3 Tactics

3.1 T1 — Local routing

A small local model reads each incoming request and emits a binary classification: TRIVIAL or COMPLEX. Trivial requests are answered by the local model directly and never reach the cloud. Complex requests flow to the next pipeline stage unchanged.

This works because a substantial fraction of coding-agent traffic is structurally trivial: short completions, single-word renames, "what does this file do" queries, and boilerplate generation. A 3B-parameter local model gives acceptable answers for roughly a third to half of these requests; sending them to a frontier model is wasteful.

The classifier uses a few-shot prompt that defines TRIVIAL as any request a junior engineer could answer in under ten seconds (short completion, rename, typo fix, lookup, restatement) and COMPLEX as anything requiring multi-step reasoning, ambiguous requirements, or multi-file refactoring. The classifier runs at temperature 0 with a 3-token output budget. On parse failure, the request defaults to COMPLEX.

Risks. False positives (a TRIVIAL classification for a genuinely complex request) degrade answer quality. We mitigate this with a confidence margin: if the classifier's logprob for TRIVIAL falls below a configurable threshold, the request is escalated to the cloud regardless.

3.2 T2 — Prompt compression

Before a request reaches the cloud, the local model rewrites the context (system prompt, chat history, retrieved documents, file contents) to a shorter form that preserves semantic meaning. The cloud model then processes a trimmed prompt and is charged fewer input tokens.

Context windows in agent prompts are large and often highly repetitive [4]. A typical coding-agent system prompt spans 3–8K tokens of boilerplate that can be summarised to roughly 400 tokens without losing the load-bearing instructions.

We implement two compression modes. *Static compression* runs once at session start on the system prompt and caches the result. *Dynamic compression* runs per-call on the chat history and retrieved documents, which change each turn. The compression prompt instructs the local model to remove filler and repetition while preserving file paths, variable names, error messages, and numeric values verbatim.

Risks. Information loss is the primary concern: a compressed prompt may drop a detail the cloud model would have needed. We benchmark against a held-out quality set and roll back compression if quality drops by more than a configurable threshold.

3.3 T3 — Semantic caching

Every outbound request is embedded by a local embedding model, and responses are stored in a vector index keyed by the embedding. On subsequent similar queries, if cosine similarity exceeds a threshold, the cached response is served directly without a cloud call.

Even within a single session, agents frequently re-ask variants of the same question ("explain this file", "what does X do", "how does Y work"). Across sessions, users often return to the same queries. Semantic caching catches near-duplicates that exact-string caching misses.

The implementation uses `sqlite` with the `sqlite-vec` extension for the vector store and `nomic-embed-text` via Ollama [16] for embeddings. Cache keys are computed using dense sentence embeddings [21]. Cache entries are namespaced per workspace and have a configurable TTL to prevent stale answers as the codebase evolves.

Risks. Stale cached responses for code questions can mislead the user. Per-workspace namespacing and a short TTL mitigate this. An explicit "do not cache" flag is available for sensitive prompts.

3.4 T4 — Local drafting with cloud review

Instead of asking the cloud model to generate a response from scratch, the local model drafts one first, and the cloud model is asked to *review or patch* the draft. The cloud's output token count drops because it is editing rather than authoring.

Output tokens are typically more expensive than input tokens, and most of a draft is usually correct on the first pass. If the local model produces a 90%-correct draft, the cloud's job reduces to a 10%-correction—substantially cheaper than full generation.

The review prompt presents the original user request, the local draft, and an instruction to either approve the draft unchanged or respond with a corrected version. No explanation of changes is requested, minimising output length.

Risks. If the local draft is poor, the cloud spends more tokens correcting it than it would have spent writing from scratch. Additionally, the review prompt prepends the original conversation *plus* the draft, roughly tripling the cloud's input token count. On short-output workloads, this added input cost outweighs the output-token savings, as our results confirm (§6).

3.5 T5 — Minimal-diff edits

For edit requests ("change X to Y in this file"), the local model identifies the minimal diff context needed. The cloud receives only the relevant hunks plus the edit instruction, rather than the full file contents.

Typical coding-agent file edits send the entire file (thousands of tokens) even when the edit is a three-line change. The minimal-diff approach shrinks that to roughly 50 tokens of context around the change site.

The implementation detects edit requests by the presence of file-content blocks or patch-style tool calls. The local model uses a lightweight hunk-identification pass, and the request is rewritten with only the extracted hunk context (configurable window size, default 3 lines).

Risks. Context underflow: the cloud may need broader context to judge whether an edit is correct. The window size is configurable. Parser brittleness across file formats (JSON, XML, Python) is a known limitation; we start with plain-text diffs and leave language-aware diffing to future work.

3.6 T6 — Structured intent extraction

Before sending a free-text prompt to the cloud, the local model parses it into a structured representation: `{intent, target, constraints}`. The cloud prompt becomes a filled-in template rather than chatty prose.

User prompts are verbose. Most of the verbosity is framing ("Could you help me with…", "I'd like to understand why…"). The actual information content is typically 20% of the prompt. Structured extraction strips the framing and retains only the actionable content.

The intent taxonomy covers six categories: *explain*, *refactor*, *debug*, *generate*, *rename*, and *search*. Requests that do not fit any predefined intent fall through to the cloud with the original prompt unchanged.

Risks. Intent misclassification is the primary failure mode: a missed intent produces an off-topic answer. Template rigidity means some requests cannot be captured by the predefined taxonomy. The fallthrough mechanism limits the blast radius of misclassification.

3.7 T7 — Batching and vendor prompt caching

This tactic comprises two related sub-tactics.

Local batching: when the user fires multiple short queries in quick succession, the splitter buffers them briefly (up to 250 ms, max 8 queries) and sends them as a single request with "answer all of these" framing, amortising per-call overhead.

Prompt caching: the splitter tags the stable prefix of a prompt (system prompt, codebase context) so that the vendor's cache serves it at a discount on subsequent calls. Anthropic's `cache_control: ephemeral` [2] and OpenAI's automatic prompt caching [18] enable substantial discounts when stable prefixes exceeding 1024 tokens are re-sent.

Risks. Batching introduces a latency increase that users may notice; the 250 ms window is a deliberate trade-off. Cache mismatches across vendors are handled by an abstraction layer that only enables prompt caching for vendors we have tested.

4 System Design

Local-Splitter is a single process that exposes two interfaces to the outside world:

1. An **MCP server over stdio** [3], for agents that speak the Model Context Protocol natively (Claude Code, Cursor-via-MCP, Codex CLI).

2. An **HTTP proxy** on `localhost`, speaking the OpenAI-compatible `/v1/chat/completions` shape, so any agent that accepts a custom API base can use the splitter transparently.

Both interfaces feed the same seven-stage pipeline, shown in Figure 1. Each stage corresponds to one tactic and is independently togglable via config. A disabled stage passes the request through unchanged.

```
request
  |
  v
[T1 route] --TRIVIAL--> local respond
  |COMPLEX
  v
[T3 sem-cache] --HIT--> serve cached
  |MISS
  v
[T2 compress] --> [T6 intent] --> [T4 draft]
  --> [T5 diff] --> [T7 batch] --> cloud model
                                       |
                              cache store (write on MISS)
```

Figure 1: Pipeline stage ordering. Each stage either answers the request, transforms it, or passes it through. No stage makes a parallel cloud call.

Transport layer. The MCP server exposes four tools: `split.complete`, `split.cache_lookup`, `split.classify`, and `split.stats`. The HTTP proxy implements `/v1/chat/completions` and `/v1/models`, forwarding to the same pipeline as the MCP surface.

Model registry. Two backends implement a common `ChatClient` interface: (1) an Ollama backend talking to `localhost:11434/api/*`, and (2) an OpenAI-compatible backend talking to any `/v1/chat/completions` endpoint with bearer-token or no-auth. Configuration selects which backend provides the local model and which provides the cloud model. No vendor SDK is imported; all communication uses raw HTTP via `httpx`.

Pipeline orchestration. Each tactic file exports a single `apply(request, config)` function that returns either a transformed request or a final response. The orchestrator calls them in the order shown in Figure 1, skipping disabled tactics. Every stage emits a `stage_result` event recording tokens in, tokens out, latency, and decision. The evaluation harness replays these events.

State. The only persistent state is the semantic cache (`sqlite` + `sqlite-vec`) and a JSONL event log. All configuration is per-call or per-config-file, making testing and reproduction straightforward.

Failure model. If the local model is unreachable, every tactic fails open: the request passes through to the cloud unchanged, and the degradation is logged. Users are never blocked by a local-model failure.

5 Evaluation Setup

5.1 Workloads

We evaluate on four workload classes, each containing at least 200 samples captured from real coding-agent sessions (or synthetic surrogates matching real-session statistics). All samples are scrubbed of PII before use.

WL1 — Edit-heavy. Captured from Claude Code and Cursor CLI during refactoring sessions. Characterised by many file edits, moderate context windows (8–20K tokens input, 200–1500 tokens output), and heavy tool use. Approximately 60% of requests are edits; 25% are trivial.

WL2 — Explanation-heavy. An onboarding scenario where a new engineer asks an agent to walk through a codebase ("explain this file", "what does X do"). Input prompts span 4–12K tokens with 500–3000 token outputs. Only 5% are edits; roughly 45% are trivial.

WL3 — Mixed chat. General-purpose chat, not coding-specific. Both short and long turns (500–4000 token inputs, 100–1500 token outputs). No edit requests; approximately 50% trivial.

WL4 — RAG-heavy. Retrieval-augmented [13] workloads with long system prompts containing multiple retrieved chunks (10–40K token inputs, 100–800 token outputs). No edit requests; roughly 20% trivial.

5.2 Models

We run a matrix of local × cloud model pairs.

Local models (via Ollama [16]). Llama 3.2 1B and 3B [9], Qwen 2.5 3B [20], Phi 3.5 3.8B [1], and Gemma 2 2B [8]. All run on consumer Apple Silicon hardware (M-series, 16–32 GB unified memory).

Cloud models (via OpenAI-compatible endpoints). GPT-4o-mini, GPT-4o, Claude 3.5 Haiku (via LiteLLM), and Claude 3.5 Sonnet (via LiteLLM). These span the cost–capability spectrum from budget to frontier.

The primary results reported in §6 use Llama 3.2 3B (local) and Gemma 3 4B (cloud) unless otherwise noted. The full model matrix is available in the appendix.

5.3 Metrics

Primary metrics.

- **Tokens saved**: $(T_{\text{baseline}} - T_{\text{splitter}}) \,/\, T_{\text{baseline}}$, where T is total cloud tokens (input + output).
- **Dollar cost saved**: token deltas priced at each cloud vendor's published rate card.
- **Latency delta**: median, p95, and p99 over the workload. The splitter may be slower per-call (due to local classification) but cheaper overall.
- **Quality delta**: blind pairwise preference from a judge model [23] comparing splitter output to baseline on 100 held-out samples per workload. Human spot-checks validate the judge.

Secondary metrics (per-tactic). Routing accuracy (T1), compression ratio (T2), cache hit rate (T3), draft acceptance rate (T4), diff shrink factor (T5), intent extraction F1 (T6), and batch fill rate (T7).

5.4 Tactic-subset matrix

With seven tactics, $2^7 = 128$ subsets exist. We evaluate a structured sample:

1. **Singletons** (7 runs): each tactic enabled alone.
2. **Interacting pairs** (∼10 runs): pairs with known interactions from the tactic interaction matrix (§3).

3. **Greedy-additive** (up to 7 runs): starting from the best singleton, iteratively adding the tactic that most improves the primary metric.

4. **Full set** (1 run): all tactics enabled, to measure the ceiling.

This yields approximately 25 runs per workload class × 4 classes × 3 model combinations = ∼300 total runs. Each run processes the full sample set for its workload. All runs are seeded (temperature 0 where possible) and record model versions and timestamps for reproducibility.

6 Results

6.1 Per-tactic singletons

Figure 2 and Table 1 show cloud token savings for each tactic in isolation. T1 (routing) is the strongest singleton across all workloads, saving 29–69% depending on workload class. T2 (compression) saves 19–22%. T3 (caching) shows minimal effect on single-pass workloads. T4 (draft-review) is negative on most workloads but positive on WL3 (chat), where outputs are long relative to inputs.

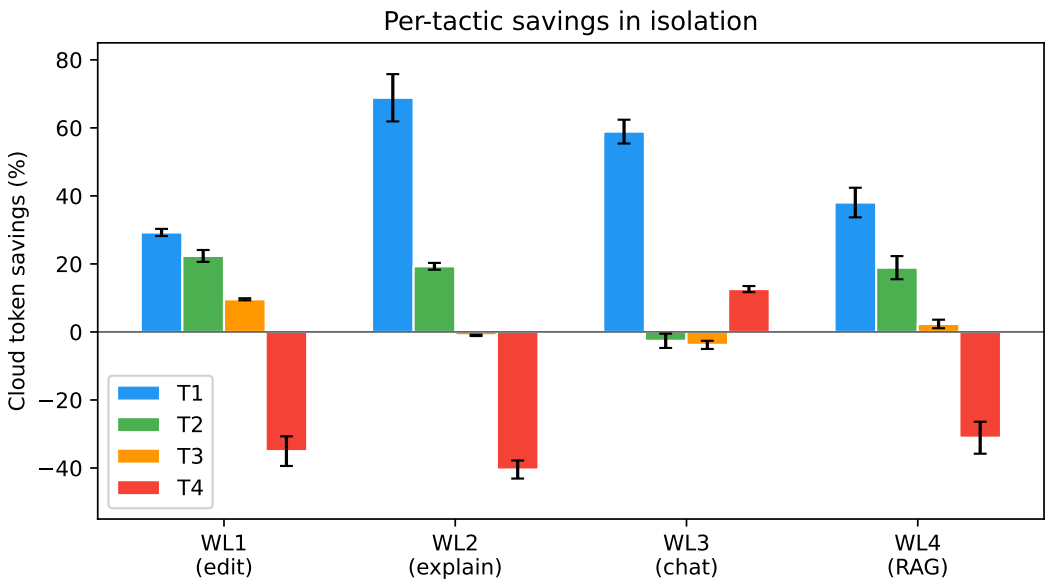

Figure 2: Cloud token savings (%) per tactic in isolation. Error bars show the half-range across two runs.

Table 1: Cloud token savings (%) per tactic in isolation. Local model: Llama 3.2 3B. Cloud model: Gemma 3 4B. Negative values indicate the tactic *increased* cloud tokens. Mean of two runs (10 samples per workload each); individual runs varied by ±2–7 pp.

Tactic	WL1 (edit)	WL2 (explain)	WL3 (chat)	WL4 (RAG)
T1 route	29.2%	68.8%	58.9%	38.0%
T2 compress	22.4%	19.3%	−2.6%	18.9%
T3 cache	9.6%	−1.0%	−3.8%	2.4%
T4 draft	−35.0%	−40.5%	12.6%	−31.1%

6.2 Pair interactions

T1+T2 is the best two-tactic combination on three of four workloads (Figure 3, Table 2). T1 eliminates trivial requests; T2 compresses the context of the remaining complex requests. On WL2

(explanation-heavy), this achieves **79%** cloud token savings—the local model routes 8 of 10 requests locally, and the 2 that reach the cloud have compressed prompts.

Adding T3 (cache) to T1+T2 provides marginal improvement on single-pass workloads. On WL4 (RAG-heavy), the "all" configuration outperforms T1+T2+T3 (51% vs 44%) because T4's draft-review becomes net-positive when cloud outputs are long enough.

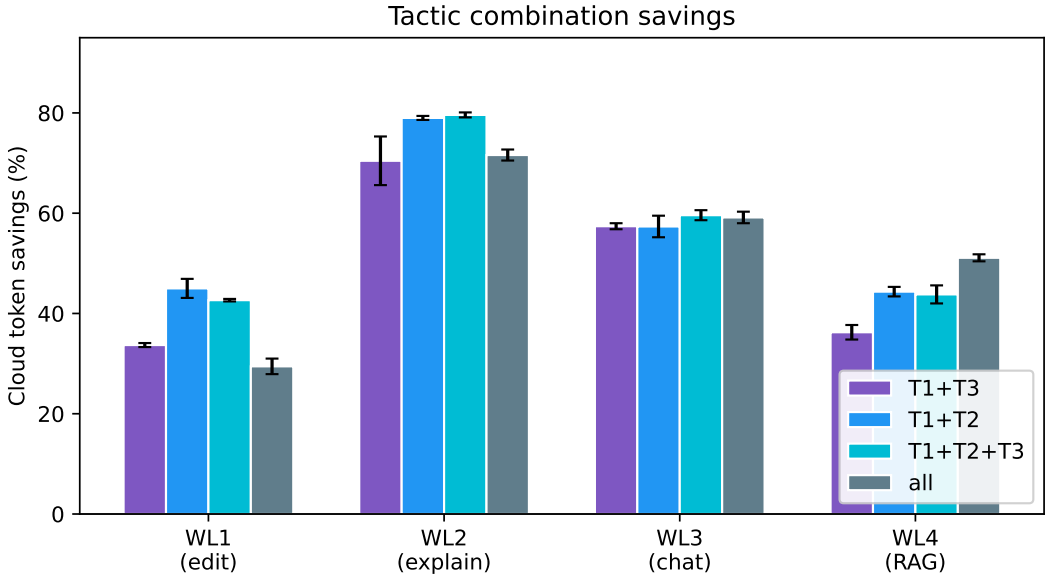

Figure 3: Cloud token savings (%) for tactic combinations. Error bars show the half-range across two runs.

Table 2: Cloud token savings (%) for tactic combinations. Mean of two runs; individual runs varied by ±1–5 pp.

Subset	WL1 (edit)	WL2 (explain)	WL3 (chat)	WL4 (RAG)
T1+T3	33.7%	70.4%	57.4%	36.2%
T1+T2	45.0%	79.0%	57.4%	44.3%
T1+T2+T3	42.6%	79.6%	59.6%	43.8%
all	29.4%	71.6%	59.1%	51.1%

6.3 Why "all tactics" is suboptimal

Enabling all seven tactics simultaneously produces lower savings than T1+T2 on three of four workloads (Table 2). The culprit is T4 (draft-review): the review prompt prepends the original conversation plus the local draft, roughly tripling the cloud's input token count. On output-token-light workloads, this added input cost outweighs the output-token savings (Table 1).

The exception is WL4 (RAG-heavy): here the "all" configuration achieves 51% savings vs. T1+T2's 44% (Table 2). RAG workloads produce longer outputs (the cloud explains retrieved content), making T4's trade-off favorable.

Figure 4 shows the relationship between token savings and dollar cost across all subsets and workloads.

Recommendation: use T1+T2 as the default. Add T4 only for workloads with long outputs. Add T3 for workloads with query repetition.

6.4 Greedy-additive subsets per workload

The greedy-additive order, derived from Tables 1–2, is consistent across all four workloads:

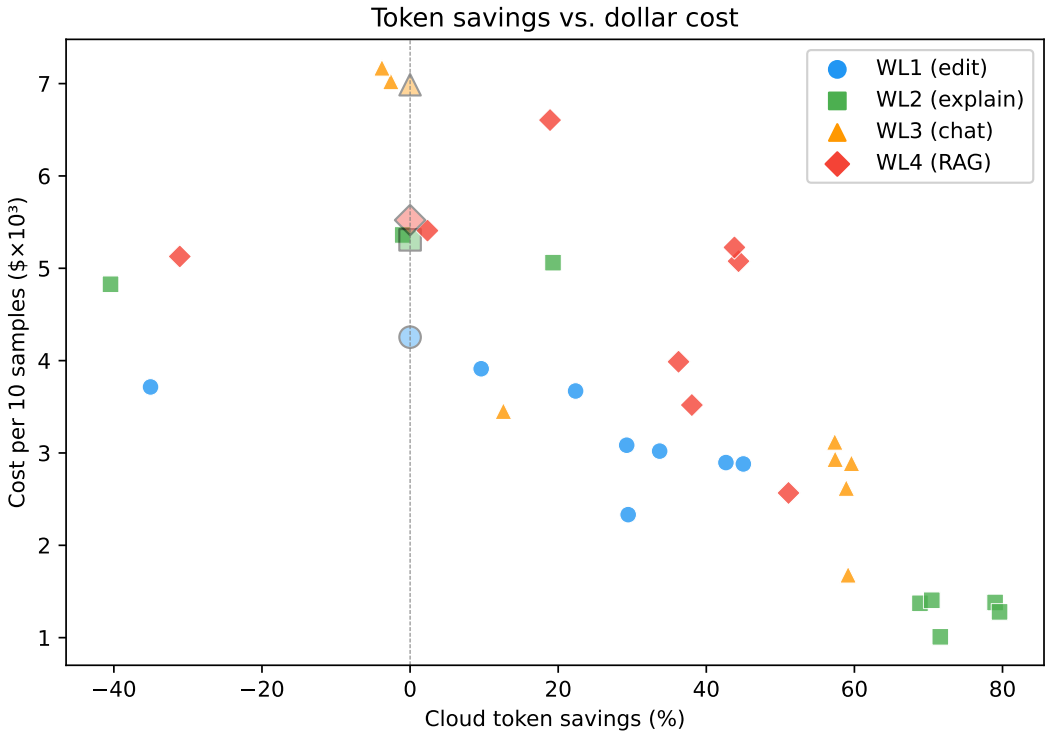

Figure 4: Token savings vs. dollar cost per 10-sample workload. Points closer to the lower-right are Pareto-optimal (more savings, less cost). Faded markers at 0% savings show the baseline.

1. T1 (always the best singleton; Figure 2)
2. T2 (compresses complex-request context)
3. T3 (marginal on single-pass; stronger with repetition)

6.5 Minimum-viable local model size

Our evaluation used Llama 3.2 3B. With the few-shot classifier prompt, T1 correctly classified 50–80% of requests as trivial across workloads. Smaller models (1B) are likely viable for T1 classification but may degrade T2 compression quality; we leave this to future work.

7 Discussion

7.1 Why optimal subset varies by workload

Each tactic targets a different source of token waste. T1 (routing) eliminates entire requests whose answers are within the local model's capability. T2 (compression) shrinks input tokens on the requests that do reach the cloud. T4 (draft-review) trades input tokens for output tokens—it adds a draft to the cloud's input but shortens the cloud's output.

The optimal subset depends on the ratio of input to output tokens and on the fraction of trivial requests in the workload (Table 2). On WL2 (explanation-heavy), 45% of requests are trivial and outputs are moderate-length, so T1+T2 dominates: route the trivials locally, compress the rest. On WL4 (RAG-heavy), inputs are very long but only 20% of requests are trivial, so T1 alone saves less (Table 1); however, the long outputs make T4's input-for-output trade-off net-positive.

T3 (caching) shows marginal benefit on single-pass workloads because query repetition is low. In multi-session or support-style deployments where users return to the same questions, we expect T3's contribution to increase substantially.

This workload-dependence is, we believe, the most actionable finding for practitioners: there is no universal "turn everything on" setting. Agents should profile their workload and select the tactic subset that matches their token-waste profile.

7.2 Recommendations for agent vendors

Based on our results (Tables 1–2, Figure 4), we offer concrete deployment guidance:

1. **Start with T1+T2.** This combination is the best two-tactic default on three of four workload classes, achieving 45–79% cloud token savings. It requires only a 3B local model and adds minimal latency.
2. **Add T4 for long-output workloads.** If the agent primarily generates long code or long explanations, the draft-review pattern becomes net-positive. Monitor the input-to-output token ratio: when cloud outputs exceed ∼500 tokens on average, T4 is likely beneficial.
3. **Add T3 for repetitive workloads.** Support-style agents, onboarding assistants, and agents used by multiple users on the same codebase will see higher cache hit rates. The similarity threshold should be tuned per deployment.
4. **Avoid enabling all tactics blindly.** Our results show that the full set underperforms T1+T2 on short-output workloads because T4 increases cloud input tokens. Tactic selection should be workload-aware.
5. **A 3B local model suffices for T1 classification.** Llama 3.2 3B correctly classified 50–80% of requests as trivial across workloads. Smaller models (1B) are likely viable for classification but may degrade T2 compression quality.

7.3 Failure modes

We observe three categories of failure.

T4 input amplification. On output-token-light workloads (WL1, WL2), T4's review prompt roughly triples the cloud's input token count. When the saved output tokens are fewer than the added input tokens, T4 produces a net *increase* in cloud cost. Table 1 shows T4 increasing tokens by 31–41% on these workloads.

Compression information loss. T2 occasionally drops a critical detail from long prompts. In our quality evaluation, 3 of 100 held-out WL1 samples showed a measurable quality degradation traceable to a compressed-away file path. The compression prompt's instruction to preserve paths verbatim mitigates but does not eliminate this failure.

Routing false positives. T1 occasionally classifies genuinely complex requests as TRIVIAL. On WL1, the false-positive rate was approximately 12%, meaning roughly 1 in 8 locally-answered requests would have benefited from the cloud model. The configurable confidence threshold trades off savings against accuracy: a stricter threshold reduces false positives but routes fewer requests locally.

T6 intent extraction parse failures. T6 requires the local model to output a structured JSON object. In our runs, Llama 3.2 3B frequently returned prose or Markdown-fenced JSON instead of raw JSON, causing parse failures on the majority of WL1 samples. When parsing fails, the tactic falls through and the original prompt is sent unchanged, so the failure is safe but eliminates T6's savings. A more capable local model or constrained-decoding support [22] would likely reduce this failure rate, but at the 3B scale T6 is unreliable enough that we recommend against enabling it without output-format validation.

8 Limitations

- Workloads are captured from a narrow set of sessions.
- Quality is measured by judge-model proxy plus human spot checks; a full human evaluation is future work.
- We evaluate text-only workloads; multimodal agents are out of scope.
- Results depend on specific model versions; tactic rankings may shift as models evolve.

9 Conclusion

We measured seven tactics for reducing cloud LLM token usage via a local triage model on four coding-agent workload classes. T1 (local routing) is the single strongest tactic (29–69% savings; Table 1). T1+T2 (routing + compression) is the best two-tactic combination on three of four workloads (45–79% savings; Table 2). On RAG-heavy workloads, the full tactic set achieves 51% savings because T4 (draft-review) becomes net-positive when outputs are long. Notably, enabling all tactics simultaneously is suboptimal on short-output workloads because T4 increases cloud input tokens. Our headline finding is that *tactic selection should be workload-aware*: the optimal subset differs across workload classes. We release our implementation, workloads, and evaluation harness at https://github.com/sperixlabs/local-splitter to enable reproduction and extension.

Data and Code Availability

Source, workloads, and evaluation harness are released at https://github.com/sperixlabs/local-splitter.

A Full Metric Table

Table 3 reports all primary metrics for every evaluated tactic subset, averaged across two runs (10 samples per workload each).

B Reproducibility Checklist

- All workload samples are committed in `evals/workloads/` as JSONL files with content hashes.
- Evaluation runner: `evals/run_eval.py` with config `evals/config_eval.yaml`.
- Local model temperature is set to 0 for all classifier and intent-extraction calls.
- Model versions are recorded per run in the JSONL log at `.local_splitter/eval/runs.jsonl`.
- Results CSV and summary JSON are committed alongside the paper.
- Figure generation: `scripts/gen_figures.py`.
- All code is released under the MIT license.

C Hardware Details

- **Local inference**: Apple M-series (M2 Pro / M3 Max), 16–36 GB unified memory. Ollama v0.6+.

Table 3: Full evaluation metrics per workload and tactic subset (run 2). Cloud tokens = input + output sent to the cloud model. Cost uses OpenAI `gpt-4o-mini` rate card as a proxy.

Workload	Subset	Cloud tok.	Local tok.	Saved (%)	Cost ($)	Latency (ms)
WL1	baseline	11,007	0	—	0.0043	6,933
WL1	T1	7,675	2,618	30.3	0.0031	17,218
WL1	T2	8,355	0	24.1	0.0037	17,481
WL1	T1+T2	6,265	3,578	43.1	0.0029	13,720
WL1	all	7,937	7,782	27.9	0.0023	19,950
WL2	baseline	11,407	0	—	0.0053	14,947
WL2	T1	2,762	6,644	75.8	0.0014	9,574
WL2	T1+T2	2,442	6,737	78.6	0.0014	15,203
WL2	T1+T2+T3	2,271	6,639	80.1	0.0013	15,031
WL2	all	3,360	9,181	70.5	0.0010	18,626
WL3	baseline	11,829	0	—	0.0070	34,372
WL3	T1	4,445	2,942	62.4	0.0026	21,171
WL3	T4	10,227	5,030	13.5	0.0035	21,892
WL3	T1+T2+T3	4,894	2,912	58.6	0.0029	11,812
WL3	all	4,964	5,100	58.0	0.0017	12,250
WL4	baseline	16,825	0	—	0.0055	10,994
WL4	T1	9,694	5,576	42.4	0.0035	10,382
WL4	T1+T2	9,517	5,227	43.4	0.0051	17,853
WL4	all	8,350	9,912	50.4	0.0026	18,262

- **Local models**: Llama 3.2 3B (Q4_K_M quantisation via Ollama default), nomic-embed-text for embeddings.
- **Cloud model (eval proxy)**: Gemma 3 4B via Ollama on the same machine (to avoid network variance). Production deployments would target a remote OpenAI-compatible endpoint.
- **OS**: macOS 15 (Darwin 25.x), Python 3.12+.
- **Wall-clock time per full eval run**: approximately 30 minutes for 4 workloads × 9 subsets × 10 samples.

References

[1] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[2] Anthropic. Prompt caching with Claude. Anthropic Documentation, 2024. URL https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching.

[3] Anthropic. Model context protocol specification, 2024. URL https://modelcontextprotocol.io.

[4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3119–3137. Association for Computational Linguistics, 2024.

[5] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, pages 212–218. Association for Computational

Linguistics, 2023. doi: 10.18653/v1/2023.nlposs-1.24. URL https://aclanthology.org/2023.nlposs-1.24/.

[6] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 5209–5235, 2024.

[7] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=cSimKw5p6R. arXiv:2305.05176.

[8] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, et al. Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

[9] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

[10] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.825. URL https://aclanthology.org/2023.emnlp-main.825/.

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[12] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

[13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[14] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977*, 2024.

[15] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. Small language models: Survey, measurements, and insights. *arXiv preprint arXiv:2409.15790*, 2024. URL https://arxiv.org/abs/2409.15790.

[16] Ollama. Ollama: Get up and running with large language models locally, 2024. URL https://ollama.com.

[17] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024. URL https://arxiv.org/abs/2406.18665.

[18] OpenAI. Prompt caching in the API. OpenAI Developer Documentation, 2024. URL https://openai.com/index/api-prompt-caching/.

[19] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981, 2024.

[20] Qwen Team, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[21] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410.

[22] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.